

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models.
(BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I, B.Harshitha 192420006 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student: B.Harshitha

Case study

QUESTIONS: Any
4

Total
Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

Object-Oriented Analysis

The Online Hotel Reservation System allows customers to search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Application of Object-Oriented Principles

1. Encapsulation

Encapsulation combines data and methods within a class and restricts direct access to the internal data. In this system, customer details, reservation information, room availability, and payment details can be kept private and accessed through appropriate methods. This provides data security and controlled access.

2. Inheritance

Inheritance allows a class to acquire common properties and methods from another class. A general User class can contain common attributes such as user ID, name, email, and password. The Customer, HotelManager, and Administrator classes can inherit these common properties while providing their own specific functions. This reduces code duplication and provides a clear class hierarchy.

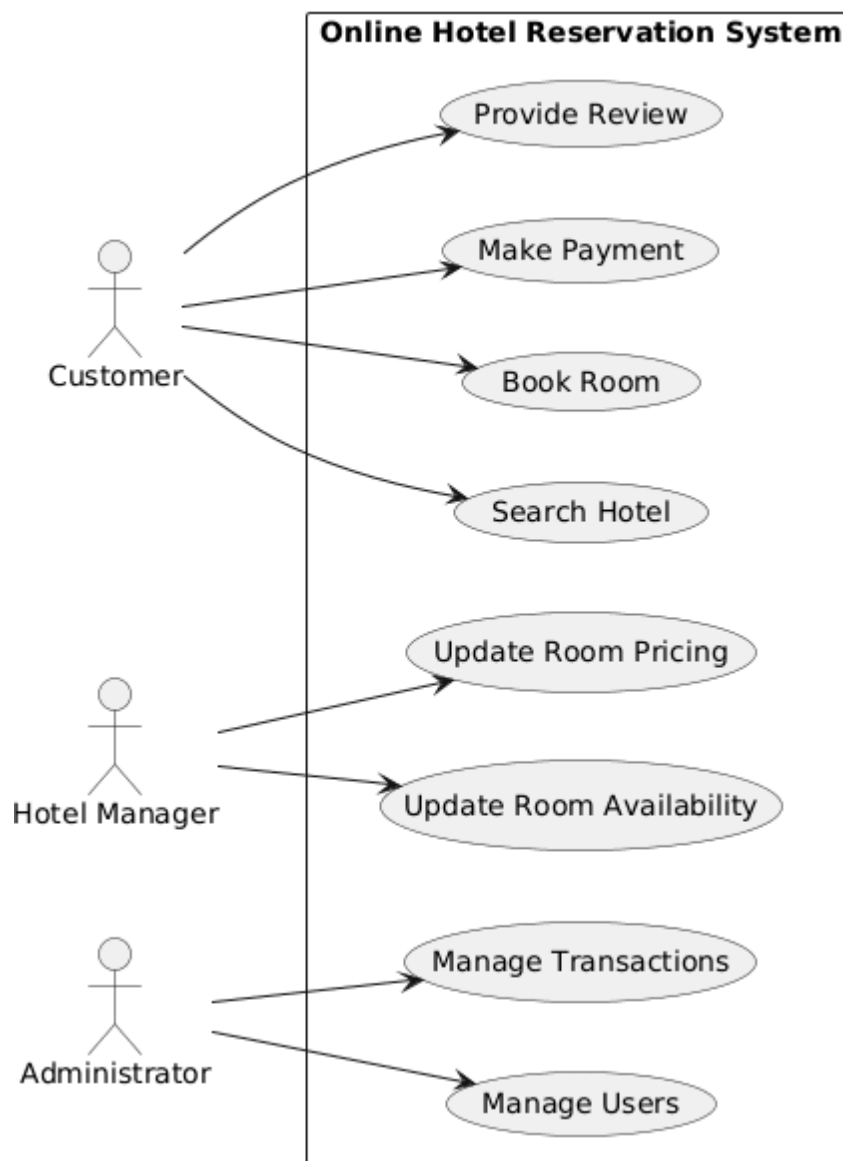


3. Polymorphism

Polymorphism allows the same method to perform different operations depending on the object. In the hotel reservation system, different payment methods such as CardPayment, UPIPayment, and NetBankingPayment can implement the same processPayment() method differently. This makes the payment system flexible and easy to extend.

UML Use Case Diagram

Online Hotel Reservation System - Use Case Diagram



Suitable SDLC Model: Agile Model

The Agile SDLC model is suitable for developing the Online Hotel Reservation System because the system contains multiple modules and its requirements may change based on customer and hotel-manager feedback.

The system can be developed incrementally through different iterations:

1. User Management
2. Hotel Search
3. Room Booking
4. Online Payment
5. Review Management
6. Administration

Justification for Agile Model

- Changing requirements: New booking, payment, or hotel-management features can be added during development.
- Continuous feedback: Customers and hotel managers can provide feedback after each iteration.
- Early testing: Individual modules can be tested before the complete system is developed.
- Incremental development: Important features can be delivered in smaller working versions.
- Easy maintenance: Individual modules can be modified or improved without affecting the entire system.

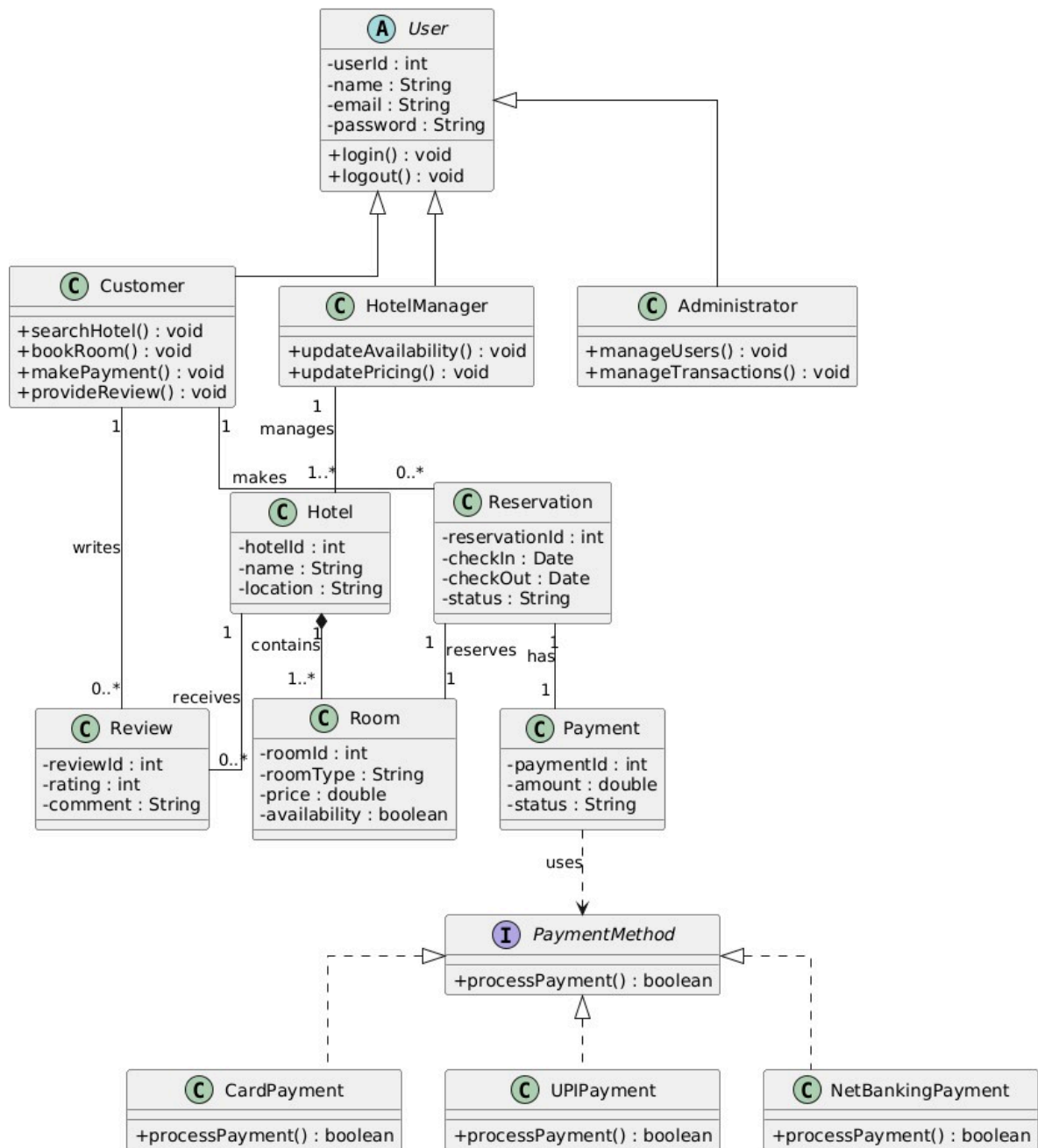
Therefore, the Agile SDLC model is the most appropriate choice because it supports iterative development, continuous feedback, early testing, and changing requirements.

Conclusion

The Online Hotel Reservation System can be effectively developed using object-oriented principles. Encapsulation provides data security, inheritance promotes code reuse, and polymorphism provides flexibility. Agile SDLC further supports the development of a flexible and maintainable system through iterative development and continuous feedback.

UML Class Diagram

Online Hotel Reservation System - Class Diagram



2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

Object-Oriented Analysis

The University Management System automates major university operations such as student admissions, course registration, examination management, and result processing. The main users of the system are students, faculty members, examination staff, and administrators.

Key Classes

The major classes identified for the system are:

- Student – manages student details, admission, and course registration.
- Course – stores course information and manages course registration.
- Faculty – manages teaching and course-related activities.
- Examination – manages examinations and marks.
- Result – processes and stores student results.

Class	Attributes	Methods
ExaminationStaff	examinationStaffId, name, email	manageExaminations(), processResults()
Administrator	adminId, name, email	manageUsers(), manageCourses()
Student	studentId, name, email	applyAdmission(), registerCourse(), viewResult()
Course	courseId, name, credits	addCourse(), updateCourse()
Faculty	facultyId, name, department	teachCourse(), enterMarks()
Examination	examId, examDate, type	scheduleExam(), conductExam()
Result	resultId, marks, grade	calculateGrade(), publishResult()
ExaminationStaff	staffId, name	manageExam(), processResult()
Administrator	adminId, name	manageUsers(), manageCourses()

Class Relationships

- A Student can register for multiple Courses.
- A Course can have multiple Students.
- A Faculty member can teach multiple Courses.
- An Examination is associated with a Course.
- A Student receives a Result for an Examination.
- ExaminationStaff manages examinations and processes results.
- Administrator manages users and courses.

These relationships provide a structured representation of how different objects interact within the university system.

Object-Oriented Approach

The system can be designed using object-oriented principles by representing students, courses, faculty, examinations, and results as separate classes. Each class contains its own data and operations, making the system modular and easier to maintain.

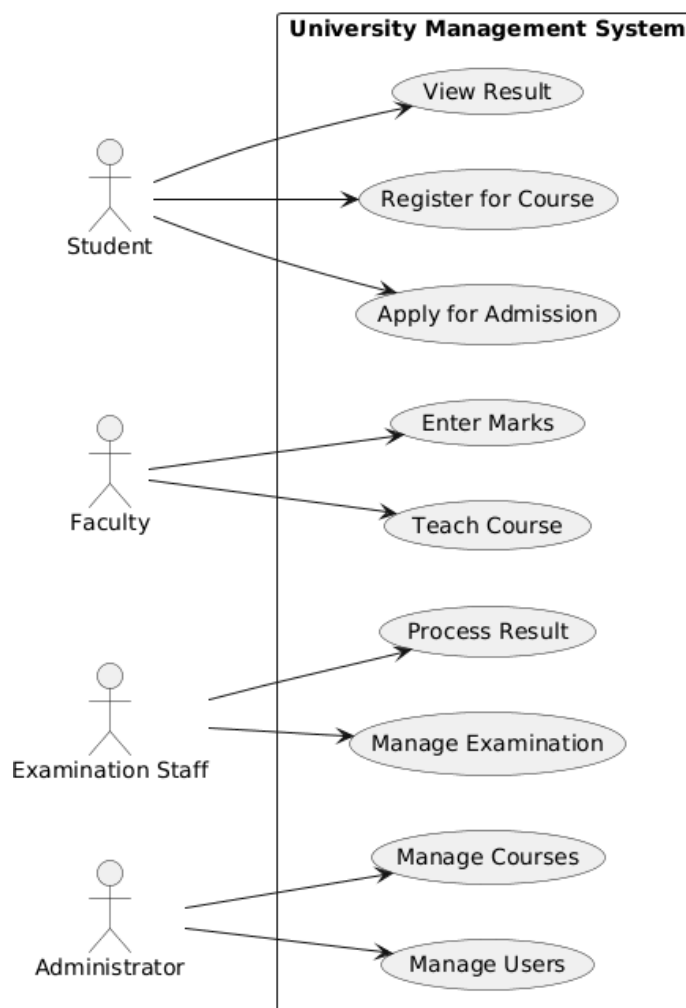
Encapsulation can be used to protect student information, marks, and result data by allowing access only through appropriate methods.

Inheritance can be applied through a common User class from which Student, Faculty, ExaminationStaff, and Administrator can inherit common attributes such as user ID, name, email, and login functions.

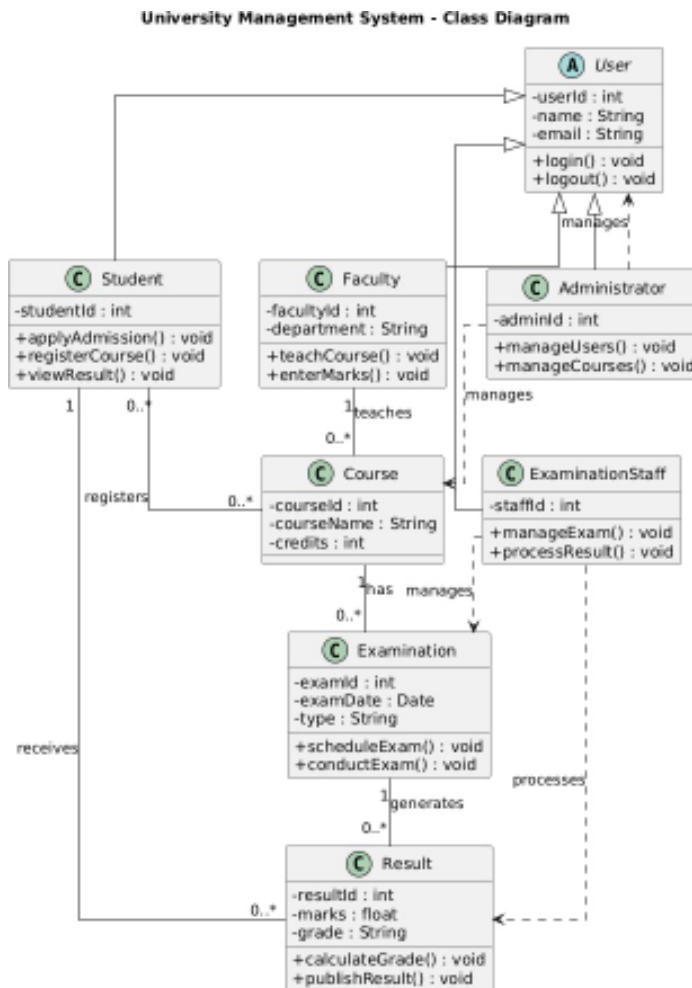
Polymorphism can be used when different users perform similar operations differently. For example, A common generateReport() operation can be implemented differently for Student, Faculty, and Administrator users, allowing each user type to receive the information relevant to their role.

UML Use Case Diagram

University Management System - Use Case Diagram



UML Class Diagram



Recommended SDLC Model: Agile Model

The Agile SDLC model is recommended for the University Management System because the system contains several independent modules and requirements may evolve during development.

The system can be developed incrementally as:

1. Student Admission
2. Course Registration
3. Faculty Management
4. Examination Management
5. Result Processing
6. Administration

Justification

- Incremental development: Each university function can be developed as a separate module.
- Changing requirements: New academic or administrative requirements can be incorporated during later iterations.
- Continuous feedback: Students, faculty, examination staff, and administrators can provide feedback during development.

- Early testing: Each module can be tested before integrating the complete system.
- Easy maintenance: Changes to one module can be made with minimal impact on other modules.

Therefore, Agile is an appropriate SDLC model because it supports incremental development, continuous feedback, early testing, and changing requirements. It helps produce an efficient and maintainable University Management System.

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

Modular Software Design Using Object-Oriented Techniques

The Online Retail Shopping Platform allows customers to browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

A modular design can be achieved by dividing the system into independent modules, where each module contains related classes and responsibilities.

Major Modules

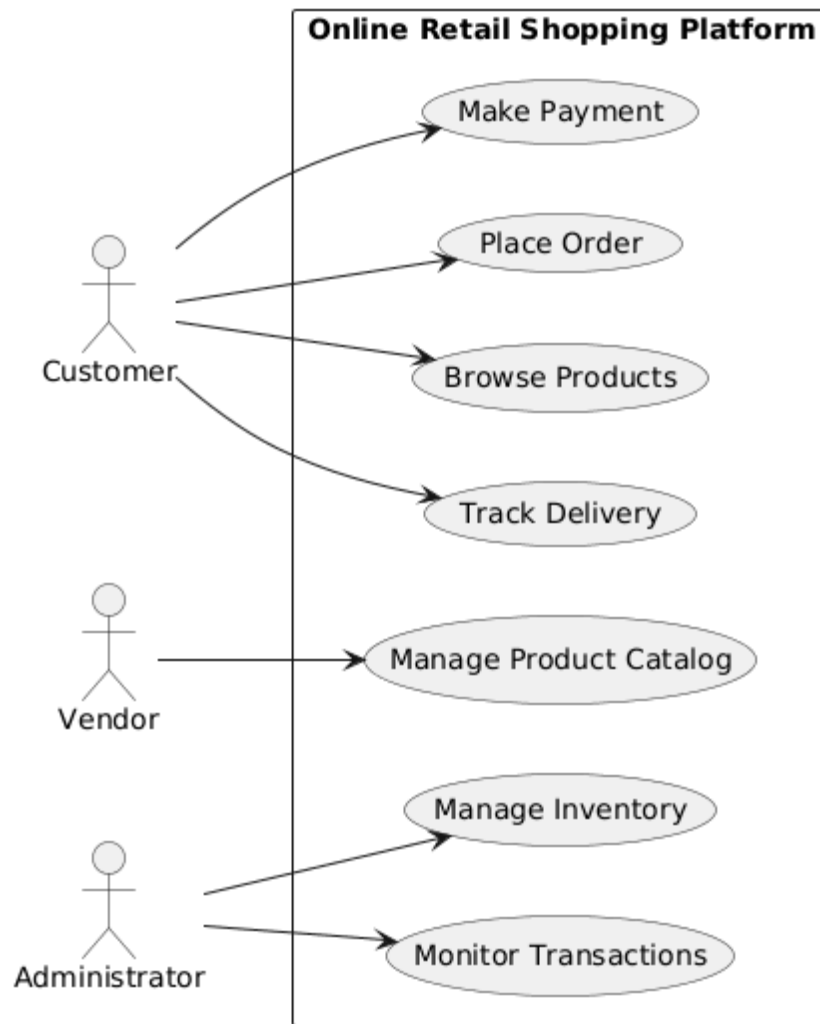
1. Customer Module
Handles customer registration, login, product browsing, and order placement.
2. Product Module
Manages product details, categories, prices, and product availability.
3. Order Module
Handles order creation, order status, and order management.
4. Payment Module
Processes online payments and maintains payment status.
5. Delivery Module
Tracks order delivery and delivery status.
6. Vendor Module
Allows vendors to manage product catalogs and product information.
7. Administration Module
Allows administrators to monitor transactions and inventory.

Object-oriented techniques support modularity by representing each major entity as a separate class with its own attributes and methods. Encapsulation keeps the data and operations of each class together, while inheritance allows common features to be reused. Polymorphism allows different payment methods or delivery methods to provide their own implementations of common operations.

This modular structure makes the system easier to develop, test, maintain, modify, and extend.

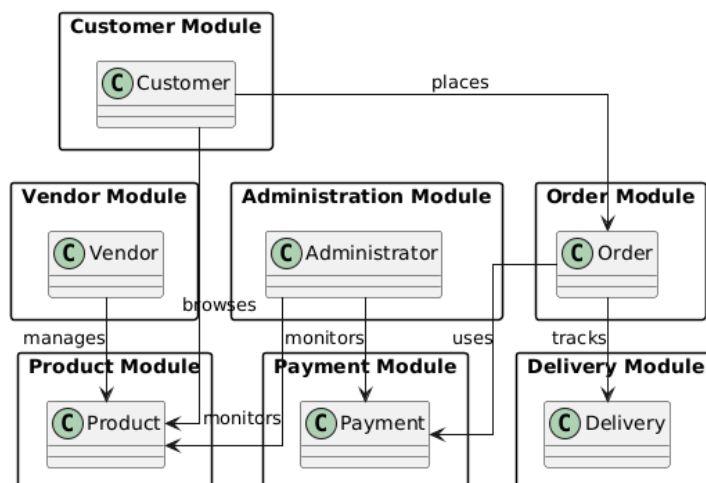
UML Use Case Diagram

Online Retail Shopping Platform - Use Case Diagram



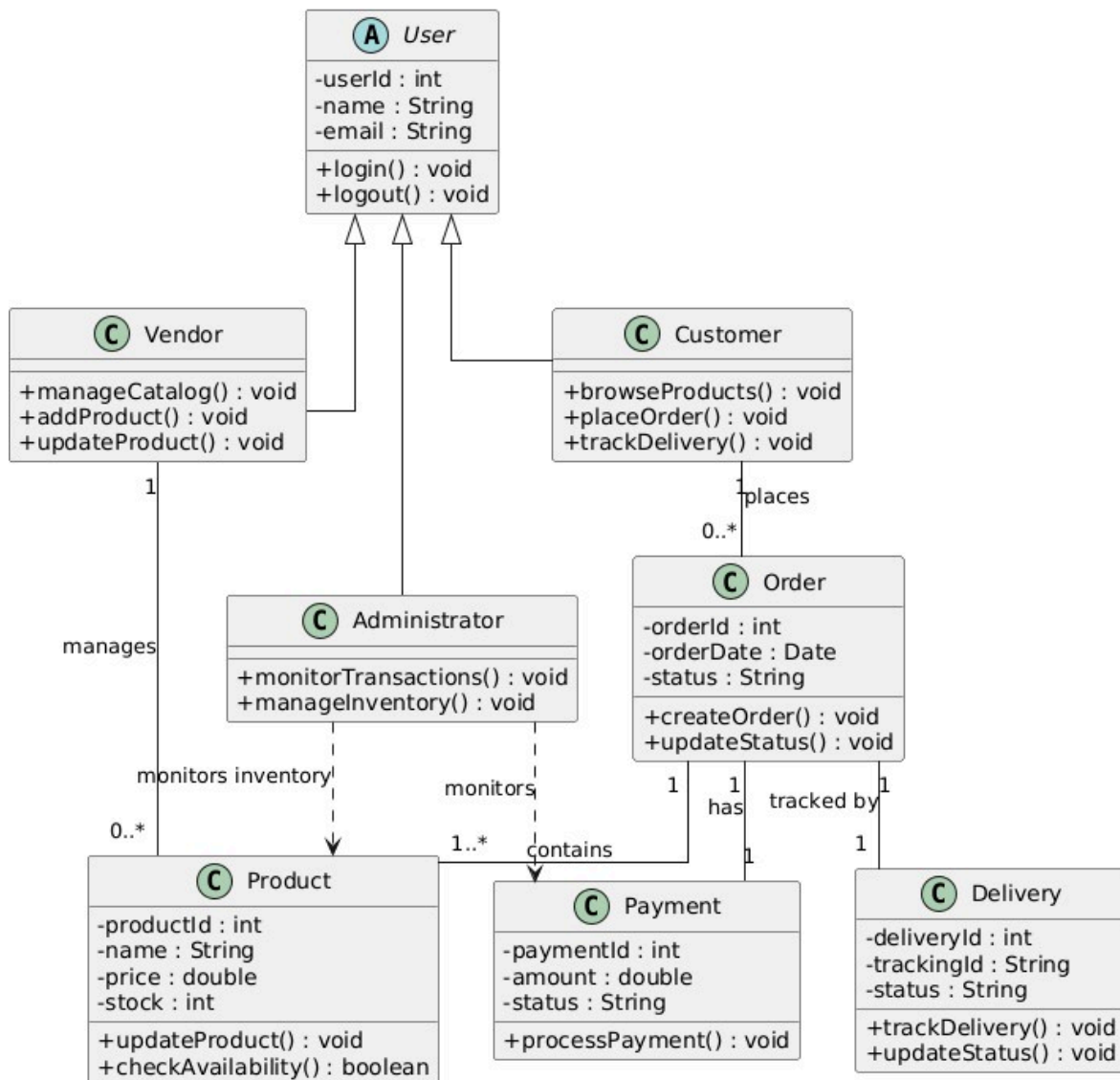
UML Modular OO Design Diagram

Online Retail Shopping Platform - Modular OO Design



UML Class Diagram

Online Retail Shopping Platform - Class Diagram



Evaluation of SDLC Models

- **Waterfall Model**

The Waterfall model follows a sequential development process in which each phase is completed before the next phase begins.

Advantage: Simple and well-structured.

Limitation: Changes in requirements are difficult to accommodate once development has progressed.

- **Spiral Model**

The Spiral model combines iterative development with systematic risk analysis.

Advantage: Suitable for identifying and managing major risks.

Limitation: It can be complex and costly for a standard e-commerce platform.

- **Agile Model**

The Agile model develops software through small iterations and allows continuous feedback and changes during development.

Advantage: Supports changing requirements, frequent testing, continuous feedback, and incremental delivery.

Limitation: Requires continuous involvement of stakeholders and effective team coordination.

Most Appropriate SDLC Model: Agile

The Agile SDLC model is the most appropriate for the Online Retail Shopping Platform.

Justification

- Changing customer requirements: New features such as payment options, offers, filters, and delivery features can be introduced during later iterations.
- Incremental development: Modules such as product browsing, ordering, payment, and delivery tracking can be developed separately.
- Continuous feedback: Customers, vendors, and administrators can provide feedback throughout development.
- Frequent testing: Each module can be tested and improved during individual iterations.
- Easy enhancement: New features can be added without redesigning the entire system.

Therefore, Agile is the most suitable SDLC model because an e-commerce platform requires flexibility, continuous improvement, frequent feedback, and incremental development.

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

Object-Oriented System Development

The Smart Waste Management System can be designed using object-oriented principles by representing important system entities as classes. The major classes include SmartBin, Sensor, CollectionVehicle, Citizen, WasteReport, and Notification.

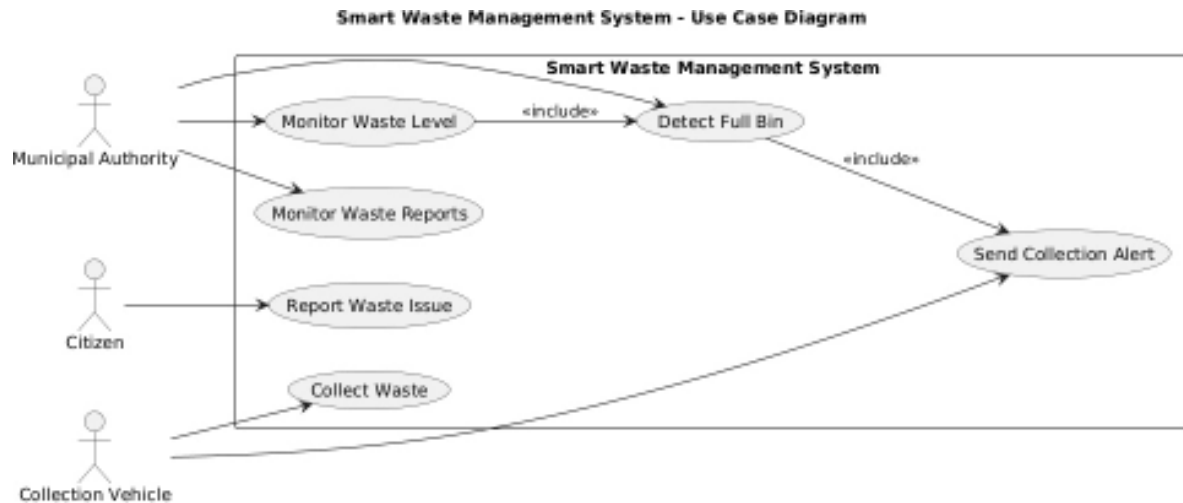
Major Classes

- SmartBin – monitors and stores the current waste level.
- Sensor – detects the waste level inside the bin.
- CollectionVehicle – receives notifications and collects waste from full bins.
- Citizen – reports waste-related problems through the mobile application.
- WasteReport – stores details of issues reported by citizens.
- Notification – sends alerts when a bin reaches its capacity.

Application of Object-Oriented Principles

- **Encapsulation:**
Sensor readings, bin capacity, waste level, and report information can be kept private within their respective classes. Public methods can be used to access or update the information. This provides controlled access and protects system data.
- **Inheritance:**
A general User class can contain common information such as user ID, name, and contact details. The Citizen class can inherit these common properties and provide additional functionality such as reportIssue().
- **Polymorphism:**
Different notification methods can implement a common sendNotification() operation. For example, mobile notifications and system alerts can provide different implementations of the same operation. This makes the system flexible for future notification methods.

UML Use Case Diagram



Suitable Life Cycle Model: Agile Model

The Agile model is best suited for the Smart Waste Management System because the system involves real-time sensor data, IoT integration, and requirements that may evolve during development.

Justification

- Real-time requirements: Sensor monitoring and automatic alerts can be developed and tested incrementally.
- IoT integration: Sensor and communication components can be integrated in stages.
- Evolving requirements: New types of sensors, notification methods, or waste-management features can be added in later iterations.
- Continuous testing: Each component can be tested regularly with real-time data.
- Continuous feedback: Municipal authorities and users can provide feedback during development.

Therefore, the Agile life cycle model is appropriate because it supports iterative development, continuous testing, evolving requirements, and gradual integration of IoT and real-time components.

UML Class Diagram

Smart Waste Management System - Class Diagram

